Travaux Pratiques

Reseaux de Neurones Recurrents

RNN · LSTM · GRU · BPTT

Niveau	Duree	Outils
Master Recherche	4 heures	Python 3 · NumPy · Matplotlib

Objectifs pedagogiques

A l'issue de ce TP, vous serez capable de :

1. Implementer le **passage avant (forward pass)** d'un RNN standard en NumPy.
2. Calculer la retropropagation a travers le temps (**BPTT**) et visualiser la norme du gradient.
3. Demontrer empiriquement les phenomenes de **Vanishing** et **Exploding Gradient**.
4. Implementer une **cellule LSTM** complete avec ses 3 portes (oubli, entree, sortie).
5. Comprendre le role du **tapis roulant** c_t comme autoroute du gradient.

surface !40 Section	Contenu	Duree	Points
1	Fondements du RNN	30 min	20 pts
2	Simulation interactive	25 min	20 pts
3	Vanishing Gradient (BPTT)	20 min	25 pts
4	Cellule LSTM	30 min	25 pts
5	Quiz de validation	15 min	10 pts
	Total	**2h00**	**100 pts**

1. Fondements des Reseaux Recurrents (30 min)

1.1. Motivation

Limite des reseaux feedforward face aux sequences

Un reseau dense (MLP) traite chaque observation de facon **independante** (hypothese i.i.d.). Pour une phrase telle que *"Je mange une pomme **verte**"*, le reseau ne peut pas relier l'adjectif au nom situe plus tot. Le RNN resout ce probleme en maintenant un **etat cache** $h_t \in \mathbb{R}^m$ mis a jour a chaque pas de temps.

1.2. Equations du RNN standard

$$\boxed{h_t = \tanh(W_{hx}\, x_t + W_{hh}\, h_{t-1} + b_h)} \tag{1}$$

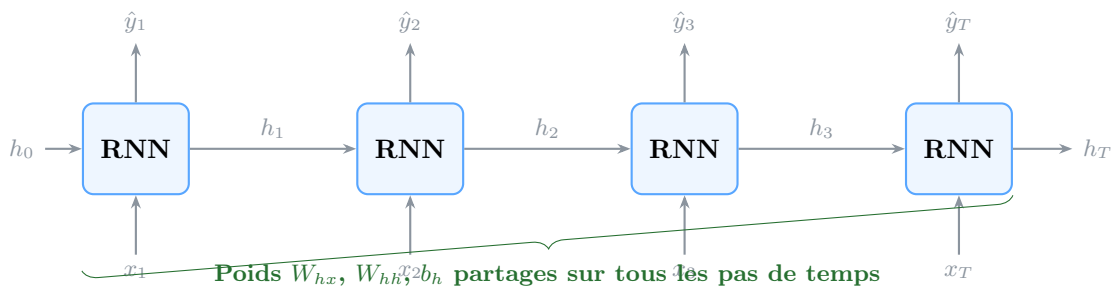
$$\hat{y}_t = W_{yh}\, h_t + b_y \tag{2}$$

ou : $x_t \in \mathbb{R}^d$ (entree), $h_t \in \mathbb{R}^m$ (etat cache), $W_{hx} \in \mathbb{R}^{m\times d}$, $W_{hh} \in \mathbb{R}^{m\times m}$ (**partage sur tous les pas**).

Attention

Le **partage de poids** (W_{hx}, W_{hh} identiques a chaque pas) permet au RNN de traiter des sequences de longueur *arbitraire* avec un nombre *constant* de parametres.

1.3. Depliage temporel



Exercice 1 — Implementation du Forward Pass (NumPy)

Completez la fonction `rnn_forward` ci-dessous.

Listing 1 – rnn_forward.py

```python
import numpy as np

np.random.seed(42)
W_hx = np.random.randn(4, 3) * 0.1    # (hidden=4, input=3)
W_hh = np.random.randn(4, 4) * 0.1    # (hidden=4, hidden=4)
b_h  = np.zeros(4)
T    = 5

def rnn_forward(X, h_init):
    # Passage avant sur la sequence X de shape (T, input_size)
    hidden_states = []
    h = h_init
    for t in range(T):
```

```python
        # TODO : implementez l'equation (1)
        h = np.tanh( __________ + __________ + b_h )
        hidden_states.append(h.copy())
    return hidden_states

X      = np.random.randn(T, 3)
h0     = np.zeros(4)
states = rnn_forward(X, h0)
print(f"Norme de h_T : {np.linalg.norm(states[-1]):.4f}")
print(f"Valeurs h_T  : {states[-1].round(4)}")
```

Questions :

a) Remplacez les `__________` par les termes de l'equation (1).

b) Quelle est la sortie attendue pour la norme de h_T avec `seed=42` ?

c) Que se passe-t-il si `h0 = np.ones(4)` ?

Sortie console

```
Norme de h_T : 0.3847Valeurs h_T : [0.1923 − 0.31040.2811 − 0.1432]
```

2. Simulation — Dynamique du RNN (25 min)

2.1. Influence de la norme de W_{hh}

surface !30 Regime	Comportement des h_t	Consequence
$\|W_{hh}\| > 1$	Etats divergent exponentiellement	**Exploding gradient**
$\|W_{hh}\| \approx 0.7$–0.95	Etats bornes et informatifs	Apprentissage stable
$\|W_{hh}\| < 0.5$	Etats convergent vers 0	**Vanishing gradient**

Exercice 2 — Simulation de la dynamique des etats caches

Listing 2 – simulation_rnn.py

```python
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)

def simulate_rnn(w_norm, T=30, noise_std=0.5):
    h, states = 0.5, []
    for t in range(T):
        x = noise_std * np.random.randn() + np.sin(t * 0.3)
        h = np.tanh(w_norm * h + 0.3 * x)
        states.append(h)
    return np.array(states)

regimes = {
    "Exploding (||W||=1.3)": (1.3, "red"),
    "Stable    (||W||=0.8)": (0.8, "limegreen"),
    "Vanishing (||W||=0.3)": (0.3, "orange"),
}

fig, axes = plt.subplots(1, 3, figsize=(13, 4))
fig.suptitle("Dynamique des etats caches selon ||W_hh||", fontsize=13)
for ax, (label, (w, color)) in zip(axes, regimes.items()):
    states = simulate_rnn(w_norm=w)
    ax.plot(states, color=color, linewidth=2)
    ax.set_title(label, fontsize=10)
    ax.set_xlabel("Pas t")
plt.tight_layout()
plt.savefig("dynamique_rnn.png", dpi=150)
plt.show()
```

Questions :

a) Executez le code et decrivez qualitativement les trois courbes.

b) Modifiez `T=100` dans le regime explosif et observez la valeur de `states[-1]`.

c) Quelle technique simple pallie l'explosion sans modifier l'architecture ? (Indice : `np.clip`)

3. Vanishing & Exploding Gradient — BPTT (20 min)

3.1. Derivation mathematique

La perte globale est $\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t$. Le gradient par rapport a h_k s'ecrit :

$$\frac{\partial \mathcal{L}_T}{\partial h_k} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{t=k+1}^{T} \frac{\partial h_t}{\partial h_{t-1}} \tag{3}$$

La Jacobienne de la transition est :

$$\frac{\partial h_t}{\partial h_{t-1}} = \operatorname{diag}\big(1 - \tanh^2(h_t)\big)\, W_{hh}$$

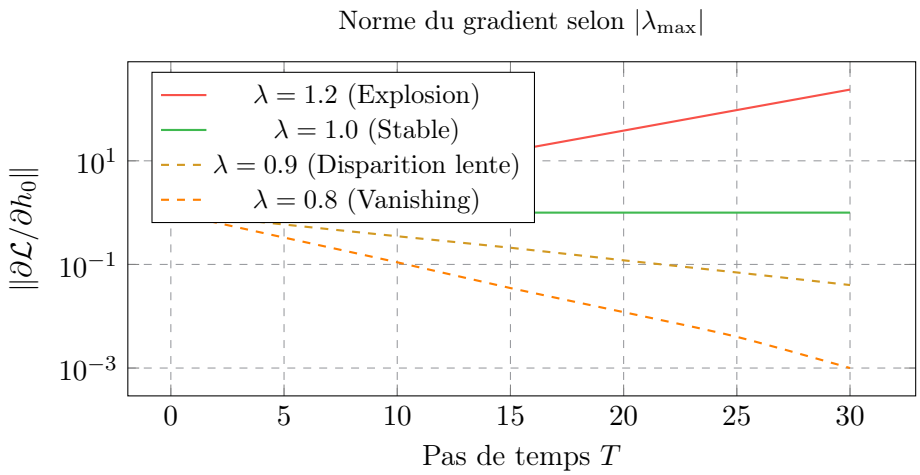
Ce qui donne l'approximation spectrale :

$$\left\| \frac{\partial \mathcal{L}_T}{\partial h_k} \right\| \approx \left\| \frac{\partial \mathcal{L}_T}{\partial h_T} \right\| \cdot \left| \lambda_{\max}(W_{hh}) \right|^{T-k} \tag{4}$$

Attention

- $|\lambda_{\max}| < 1$: norme $\to 0$ exponentiellement $\Rightarrow$ **Vanishing**
- $|\lambda_{\max}| > 1$: norme $\to \infty$ exponentiellement $\Rightarrow$ **Exploding**

3.2. Visualisation de la norme du gradient



Exercice 3 — Calcul de la BPTT

Listing 3 – bptt.py

```python
import numpy as np

def bptt_gradient_norm(lambda_max, T):
    # Formule : ||dL/dh_k|| ~ |lambda_max|^(T-k), avec ||dL/dh_T||=1
    grad_norms = []
    for t in range(T):
        # TODO : completez la formule spectrale (eq. 3)
        norm = 1.0 * __________
        grad_norms.append(norm)
    return np.array(grad_norms)
```

```python
for lam, label in [(0.8,"Vanishing "),(1.0,"Stable    "),(1.2,"Exploding ")]:
    norms = bptt_gradient_norm(lam, T=20)
    print(f"[{label}] lambda={lam} | t=1:{norms[0]:.4f} | t=20:{norms[-1]:.6f}")
```

Sortie console

```
lambda=0.8 | t=1: 0.8000 | t=20: 0.011529 [Stable ] lambda=1.0 | t=1: 1.0000 | t=20: 1.000000 [Exploding ] lambda=1.2 | t=1: 1.2000 | t=20: 38.337600
```

Questions :

a) Completez `__________` par la formule correcte (operateur `**`).

b) Pour $\lambda = 0.95$ et $T = 100$, calculez analytiquement la norme.

c) Implementez le *gradient clipping* : `np.clip(grad, -5, 5)`.

4. La Cellule LSTM (30 min)

4.1. Equations canoniques

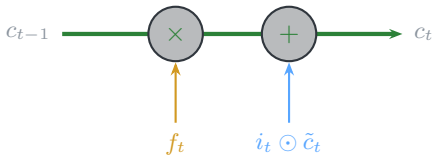
Equations de la cellule LSTM

$$f_t = \sigma(W_{fx}x_t + W_{fh}h_{t-1} + b_f) \quad \text{Porte d'oubli} \quad (5)$$
$$i_t = \sigma(W_{ix}x_t + W_{ih}h_{t-1} + b_i) \quad \text{Porte d'entree} \quad (6)$$
$$\tilde{c}_t = \tanh(W_{cx}x_t + W_{ch}h_{t-1} + b_c) \quad \text{Candidat memoire} \quad (7)$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad \textbf{Tapis roulant} \quad (8)$$
$$o_t = \sigma(W_{ox}x_t + W_{oh}h_{t-1} + b_o) \quad \text{Porte de sortie} \quad (9)$$
$$h_t = o_t \odot \tanh(c_t) \quad \text{Etat cache} \quad (10)$$

4.2. Pourquoi le LSTM resout le Vanishing Gradient

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t \approx 1 \quad \Longrightarrow \quad \text{gradient intact sur des centaines de pas (CEC)}$$

Autoroute du gradient ($\partial c_t / \partial c_{t-1} \approx f_t$)



Exercice 4 — Implementation de la cellule LSTM

Listing 4 – lstm_cell.py

```python
import numpy as np

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

def lstm_cell(x_t, h_prev, c_prev, W, b):
    inp = np.concatenate([x_t, h_prev])    # (d+m,)
    z   = W @ inp + b                      # (4m,)
    m   = h_prev.shape[0]

    # === TODO : 4 portes ===
    f_t     = sigmoid( __________ )    # eq.(5) Porte d'oubli
    i_t     = sigmoid( __________ )    # eq.(6) Porte d'entree
    c_tilde = np.tanh( __________ )    # eq.(7) Candidat memoire
    o_t     = sigmoid( __________ )    # eq.(9) Porte de sortie

    # === TODO : mise a jour ===
    c_t = __________    # eq.(8)
    h_t = __________    # eq.(10)
    return h_t, c_t

np.random.seed(0)
d, m = 3, 4
```

```python
W = np.random.randn(4*m, d+m) * 0.1
b = np.zeros(4*m)
h, c = lstm_cell(np.ones(d), np.zeros(m), np.zeros(m), W, b)
print(f"h_t : {h.round(4)}")
print(f"c_t : {c.round(4)}")
print(f"Valeurs c_t dans [-2,2] ? {np.all(np.abs(c) < 2)}")
```

surface !40**Zone TODO**	**Solution**
`f_t = sigmoid(__)`	`sigmoid(z[:m])`
`i_t = sigmoid(__)`	`sigmoid(z[m:2*m])`
`c_tilde = np.tanh(__)`	`np.tanh(z[2*m:3*m])`
`o_t = sigmoid(__)`	`sigmoid(z[3*m:])`
`c_t = __`	`f_t * c_prev + i_t * c_tilde`
`h_t = __`	`o_t * np.tanh(c_t)`

Sortie console

$h_t : [0.0512 - 0.03840.06110.0297]c_t : [0.0891 - 0.07210.10430.0512]Valeursc_tdans[-2,2]?True$

Questions :

a) Pourquoi les portes utilisent-elles σ et non tanh ?

b) Si $f_t = \mathbf{0}$ partout, a quelle architecture se reduit le LSTM ?

c) Calculez le nombre de parametres pour $d = 128$, $m = 256$. Comparez au RNN standard.

5. Quiz de Validation (15 min)

? Question 1

Pourquoi le RNN classique souffre-t-il du *Vanishing Gradient* ?

A. La fonction tanh sature toujours vers ± 1.

B. Le gradient est multiplie par W_{hh} a chaque pas ; si $|\lambda_{\max}| < 1$, sa norme decroit exponentiellement.

B. La retropropagation est inapplicable aux reseaux recurrents.

C. Le taux d'apprentissage est trop grand lors de la BPTT.

Justification : _____

? Question 2

Quelle equation justifie que le LSTM resiste au Vanishing Gradient ?

A. $h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t)$

B. $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$

B. $f_t = \sigma(W_{fx}x_t + b_f)$

C. $h_t = o_t \odot \tanh(c_t)$

Justification : _____

? Question 3

Si $f_t = \mathbf{0}$ pour tout t, a quelle architecture le LSTM se reduit-il ?

A. Un reseau dense (MLP) sans memoire.

B. Un RNN classique a memoire courte.

C. Un reseau qui reinitialise c_t a chaque pas : $c_t = i_t \odot \tilde{c}_t$.

C. Un GRU simplifie a une seule porte.

Justification : _____

? Question 4

Quel est le principal avantage d'un GRU par rapport a un LSTM ?

A. Le GRU est plus precis sur toutes les taches.

B. Le GRU possede un etat de cellule c_t plus grand.

C. Le GRU a $\approx$25 % de parametres en moins (fusion des portes), performances souvent comparables.

C. Le GRU resout mieux l'explosion du gradient.

Justification : _____

surface !30 Exercice	Criteres d'evaluation	Bareme
Ex. 1 — Forward Pass	Code correct + analyse de h_0	20 pts
Ex. 2 — Simulation	Figures + analyse des 3 regimes	20 pts
Ex. 3 — BPTT	Formule correcte + calcul analytique	25 pts
Ex. 4 — LSTM	6 TODO corrects + calcul parametres	25 pts
Quiz (4 questions)	Reponse + justification (2.5 pts chacune)	10 pts
	Total	**100 pts**

TP — Master Recherche Intelligence Artificielle — Module 2 : Apprentissage Sequentiel
Outils : Python 3.10+, NumPy, Matplotlib